Project Definition & Development Contract

Building a Vulkan Renderer in C++20

De Montfort University | BSc (Hons) Games Production

Mohamed Deeq Mohamed | P2884884 | 28th March, 2026

Version 13.0: Vulkan 1.3 Dynamic Rendering; vk-bootstrap throughout; MoSCoW prioritisation; no `VkRenderPass`, no `VkFramebuffer`

1. Student Name:	Mohamed Deeq Mohamed	**2. P-Number:**	P2884884
3. Project Title:	Building a Vulkan Renderer in C++20	**2a. Deadline:**	1 June 2026
4. Supervisor(s):	Salim Hashu / Dr Conor Fahy	**2b. Start:**	30 March 2026

This document serves as both the Project Definition and a formal Development Contract. The objectives stated here constitute a binding contract; the final deliverable will be evaluated, in part, against them.

5. Introduction (max. 100 words)

This project develops a real-time 3D renderer in Vulkan 1.3 and C++20, targeting the explicit, low-level GPU programming paradigm demanded by contemporary graphics and neural rendering research. Initialisation is handled through **vk-bootstrap**; all rendering is implemented via **Dynamic Rendering**, eliminating `VkRenderPass` and `VkFramebuffer` from the codebase entirely. The work constitutes a deliberate engineering foundation for a planned PhD in neural rendering at De Montfort University commencing October 2026, with Gaussian splat rendering serving as a bridging milestone between classical rasterisation and neural scene representation.

6. Aims (max. 100 words)

This project develops a principled understanding of how contemporary GPU hardware processes and rasterises geometry by designing and implementing a Vulkan 1.3 renderer in C++20. Decisions across synchronisation strategy, resource binding, and shading models are each grounded in explicit technical justification; architectural clarity and extensibility are treated as first-class outcomes. Vulkan initialisation is abstracted through vk-bootstrap, consistent with professional practice. The resulting codebase is designed as the intended engineering foundation for postgraduate research in neural rendering, Gaussian-based scene reconstruction, and GPU-accelerated rendering at De Montfort University.

6a. State of the Art

Modern real-time rendering is grounded in physically based shading models that approximate light transport using microfacet BRDFs. The Cook–Torrance formulation and the Disney principled BRDF [1] established surface roughness, metalness, and view-dependent specular response as physically interpretable parameters, and remain the de facto shading basis for both offline renderers and real-time game engines.

At the API level, Vulkan 1.3's dynamic rendering path removes the requirement for explicit

render pass and framebuffer objects [2], and is now the recommended baseline for new rendering back-ends. It preserves full pipeline control while eliminating substantial structural overhead from the API surface.

Neural rendering has emerged as a defining research direction in computer graphics. Neural Radiance Fields (NeRF) [3] represent scenes as continuous volumetric functions optimised from multi-view images, enabling high-fidelity novel view synthesis; however, surveys on the field identify persistent trade-offs among reconstruction quality, training cost, and real-time inference performance [4].

3D Gaussian Splatting (3DGS) [5] directly addresses these trade-offs through an explicit, GPU-friendly scene representation: millions of anisotropic 3D Gaussian primitives are rasterised to screen-space ellipses and alpha-composited in real time, achieving radiance field rendering at 1080p without volumetric marching. Follow-up surveys underscore the demand for rendering back-ends capable of integrating classical rasterisation, physically based shading, and neural scene representations [6]. This project positions a Vulkan 1.3 renderer as such a back-end, with Gaussian splat rendering constituting a direct pilot for the planned PhD research programme.

7. Objectives (max. 200 words)

Each objective is colour-coded by MoSCoW priority (see Appendix B): Must Have Should Have Could Have

1. **Baseline Vulkan Pipeline (Week 2), *Must Have*:** Establish a Vulkan 1.3 environment via vk-bootstrap and render a coloured triangle using Dynamic Rendering. Neither `VkRenderPass` nor `VkFramebuffer` shall appear anywhere in the project.
2. **Rotating 3D Geometry (Week 3), *Must Have*:** Implement vertex and index buffers via staging buffer upload, attach a VMA-managed depth image dynamically, and supply MVP transformation data through a UBO and descriptor set.
3. **Textured OBJ Model (Week 5), *Must Have*:** Load a mesh via `tinyobjloader` and apply a texture through a combined image sampler, employing `synchronization2` for all layout transitions.
4. **Technical Report and Renderer Baseline (Week 9), *Must Have*:** Deliver a renderer with zero validation errors and a reproducible CMake build, accompanied by a technical report demonstrating design rationale, architectural understanding, and critical reflection on the project's research aims.

5. **PBR Shading (Week 7), *Could Have*:** Implement the Cook–Torrance BRDF [7] (GGX normal distribution, Smith geometry term, Schlick Fresnel approximation) with Dear ImGui runtime material controls. The minimum viable implementation is Lambertian diffuse shading.
6. **Gaussian Splatting (Post Week 7), *Could Have*:** Render a `.ply` Gaussian splat scene as a PhD research pilot, following the method of Kerbl et al. [5].
7. **XR Exploration (If time permits), *Could Have*:** Investigate OpenXR integration or document it as a direction for further work.

8. Deliverables (max. 100 words)

- A real-time Vulkan 1.3 renderer in C++20, employing Dynamic Rendering throughout, with OBJ loading and texture mapping as the core feature set. Mipmapping, PBR shading, Dear ImGui controls, and Gaussian splat rendering (`.ply` input) are pursued as extensions beyond the assessed minimum.
- A public GitHub repository (`Raiju-Deeq/FYP-Vulkan-Renderer`) with a sustained commit history, reproducible via `CMakePresets.json` on both Linux and Windows using vcpkg.

- A technical report covering architectural design, pipeline implementation, and critical reflection on the project's research aims and outcomes.

9. Schedule of Activities

Week	Focus	Key Tasks
1	Setup and toolchain	Configure CMake with vcpkg (vk-bootstrap); install the Vulkan SDK, validation layers, and RenderDoc; initialise the GitHub repository.
2	Triangle render	Complete vk-bootstrap initialisation; build the SPIR-V pipeline; implement the Dynamic Rendering command sequence; write the layout transition helper.
3	Rotating cube	Create vertex and index buffers; implement staging buffer uploads; set up the VMA depth image; configure UBO and descriptor sets; apply MVP transforms.
4	Consolidation and report	Conduct an RAII audit; begin writing the report introduction and architecture sections.
5	Textured OBJ model	Integrate `tinyobjloader`; set up the combined image sampler; apply `synchronization2` barriers; generate the mipmap chain if time permits.
6	Report (parallel)	Write pipeline and shading model sections of the report; conduct BRDF background research.
7	PBR shading *(Could Have)*	Implement GGX NDF, Smith G, and Schlick Fresnel; build the Dear ImGui material panel; create Blender reference comparisons.
7–8	Gaussian splatting *(Could Have)*	Parse `.ply` files; allocate GPU splat buffers; implement depth sorting, ellipse projection, and alpha compositing.
8	Renderer polish	Implement the camera; handle swapchain resizing; achieve zero validation errors; perform RenderDoc frame captures.
9	Final submission	Complete the report; tidy the GitHub repository; verify the CMake build on both target platforms; obtain supervisor sign-off.

10. Requirements

Software: Vulkan SDK 1.3, C++20 (GCC/MSVC/Clang), CMake 3.25+, vcpkg, vk-bootstrap, Vulkan Memory Allocator, tinyobjloader, tinyply, Dear ImGui, GLM, stb_image, spdlog, shaderc/glslc, RenderDoc, JetBrains CLion, Visual Studio 2022, GitHub, and Blender (Cycles).

Hardware: A GPU with Vulkan 1.3 support (AMD or NVIDIA discrete). Windows 10/11 or Linux (Arch). VR headset access is a requirement only if the XR stretch goal is reached; it is not a hard dependency of the assessed submission.

11. Research Hypothesis

"A Vulkan 1.3 renderer built against the raw API, using industry-representative abstractions at the initialisation layer only, constitutes a sufficient engineering foundation for postgraduate research in neural rendering: the design decisions required to implement explicit synchronisation, GPU memory management, and a Gaussian splat rendering pipeline develop the architectural understanding that the neural rendering research domain demands, and produce a codebase directly extensible to that domain."

This proposition is evaluated empirically through the renderer's architecture and through the

critical reflection section of the technical report, which assesses whether each major design decision can be mapped to a corresponding research competency or extensibility requirement.

Supervisory Agreement & Signatures

Party	Signature	Date
12. Student: Mohamed Deeq Mohamed (P2884884)	_____	_____
13. Supervisor: Salim Hashu	_____	_____
Supervisor: Dr Conor Fahy	_____	_____

14. Date: _____

Appendix A: Grade Band Mapping & Milestone Contract

The table below defines the minimum milestone completion required for each grade band. **Could Have milestones (PBR, Gaussian splatting, XR) are not required for any grade band.** Failing to attempt them will not penalise the assessment outcome. Should Have items (camera, resize handling, mipmapping, RenderDoc analysis) are expected to be delivered and will strengthen any submission, but are not formally binding.

Grade	Mark	Minimum	Rationale
Fail	<40%	Below Obj. 1	No functional Vulkan output.
Third	40–49%	Objectives 1–2 + Report intro and architecture	Working pipeline with triangle and rotating cube.
2:2	50–59%	Objectives 1–3 + Report covering pipeline	Textured OBJ model with a functional texture pipeline.
2:1	60–69%	Objectives 1–3 + strong report + renderer polish	Should Have items delivered; clear design rationale in the report.
First	**70%+**	**Objectives 1–4 + technical report**	**High code quality; critical reflection; design decisions justified against research aims. PBR and stretch goals are not required.**

Important: A First Class Honours outcome is achievable through Objectives 1 to 4 alone: baseline pipeline, rotating geometry, textured OBJ model, and a technical report demonstrating architectural understanding and critical reflection. Could Have objectives are pursued for personal and research benefit only; they are not assessed differentiators.

#	Milestone	Priority	Grade Band	Target	Sign-off
1	Baseline Vulkan pipeline + triangle	Must Have	Third (40%+)	Week 2	_____
2	Rotating 3D geometry	Must Have	2:2 (50%+)	Week 3	_____
3	Textured OBJ model	Must Have	2:1+ (60%+)	Week 5	_____
4	Renderer baseline + technical report	Must Have	First (70%+)	Week 9	_____
5	PBR shading	Could Have	Not graded	Week 7	_____
6	Gaussian splatting (`.ply`)	Could Have	Not graded	Post Obj. 3	_____
7	XR/VR exploration	Could Have	Not graded	If time	_____

Appendix B: MoSCoW Prioritisation

Must Have

- Vulkan 1.3 Dynamic Rendering throughout. Neither `VkRenderPass` nor `VkFramebuffer` shall appear anywhere in the codebase.
- Baseline triangle render via `vkCmdBeginRendering` (Objective 1).
- Rotating 3D geometry with VMA depth image, UBO, and MVP transforms (Objective 2).
- Textured OBJ model with a combined image sampler and `synchronization2` layout transitions (Objective 3).
- RAII-compliant C++20 codebase with zero validation errors on both startup and shutdown.
- Reproducible CMake build on Linux and Windows via vcpkg.
- Public GitHub repository with a sustained commit history.
- A technical report demonstrating design rationale, architectural understanding, and critical reflection (Objective 4).

Should Have

- Mipmap chain generation via `vkCmdBlitImage`.
- Interactive orbit or first-person camera.
- Swapchain resize handling (`VK_ERROR_OUT_OF_DATE_KHR`).
- Dear ImGui overlay for runtime controls.
- RenderDoc frame capture analysis with GPU timing data.
- Per-milestone evidence packages (annotated screenshots, validation logs, written notes).

Could Have (Not Required for Any Grade Band)

- **PBR shading:** Cook–Torrance BRDF [7] (GGX normal distribution, Smith geometry term, Schlick Fresnel approximation) with Dear ImGui material controls. The minimum viable implementation is Lambertian diffuse. This objective is pursued because

a working command of shading mathematics is directly pertinent to PhD research in neural rendering, where differentiable shading models are an active area of investigation.

- **Gaussian splat rendering:** Ingest a `.ply` file, sort and project 3D Gaussians to screen-space ellipses, and alpha-composite the result, following the method of Kerbl et al. [5]. This objective constitutes a direct engineering pilot for the planned PhD research programme in Gaussian-based neural rendering at De Montfort University and will not be commenced until Objectives 1 through 3 have been formally signed off.
- **XR/AR exploration via OpenXR:** Investigate OpenXR integration within the renderer. Vulkan is the natural XR back-end and Dynamic Rendering is compatible with per-eye render structures. A full implementation is not expected within the project timeline; however, substantive progress or a detailed further-work section in the report would constitute a meaningful research contribution. This objective is pursued only after Gaussian splatting is complete or formally deferred.

Won't Have (Explicitly Out of Scope)

This is a **renderer**, not a game engine. The following features are explicitly outside scope: physics simulation, entity-component systems, scene graphs, scripting runtimes, audio, level editor tooling, networking, shadow mapping, deferred shading, a full ray tracing pipeline, mobile or console Vulkan, NeRF training or inference, and multi-GPU rendering.

Appendix C: Risk Register

Risk	Likelihood	Impact	Mitigation
Report deprioritised in favour of coding	High	High	Report writing begins in Week 4. A challenges and reflection log is maintained throughout development and incorporated into the report rather than written retrospectively.
Vulkan synchronisation errors consuming development time	High	Medium	Validation layers remain enabled at all times. `synchronization2` is used for all barriers. RenderDoc is employed from Milestone 1 onwards.
Gaussian splatting scope creep displacing core polish	Medium	High	Objective 6 is not commenced until Objectives 1 through 3 have been signed off. Any time pressure on Objective 4 immediately suspends all stretch work.
Image layout transition errors under Dynamic Rendering	Medium	Medium	A reusable `transitionImageLayout` helper is written at Objective 1 and applied consistently thereafter.
vk-bootstrap incompatibility on DMU lab GPU drivers	Low	Medium	Full fallback control is available via vk-bootstrap selectors. The `uni-debug` preset is verified on a lab machine before Objective 1 sign-off.
CMake/vcpkg build failures across platforms	Low	Medium	Both presets are verified on clean environments before submission.

Key: High Medium Low

Appendix D: Student Commitments

- Milestone evidence is presented at each weekly check-in.

- Report writing begins no later than **Week 4**. A challenges and reflection log is maintained throughout development and forms the basis of the report's critical reflection section.
- The vk-bootstrap initialisation sequence is documented in the report to demonstrate architectural understanding regardless of the abstraction layer employed.
- **Objective 6 (Gaussian splatting) is not commenced until Objectives 1 through 3 have been signed off.** Time pressure on Objective 4 immediately pauses all stretch work without exception.
- Scope additions are discussed with supervisors before implementation.
- Work is tracked in the public GitHub repository with regular, descriptive commits and the use of issues where appropriate.
- Project management is conducted through a dedicated Jira Software project configured with epics mirroring Objectives 1 through 7 to maintain accountability and progress visibility.

Supervisory responsibilities: weekly milestone review (approximately 30 minutes); sign-off on evidence packages; written report feedback no later than one week before the submission deadline.

References

[1] B. Burley, "Physically based shading at Disney," *SIGGRAPH Course: Practical Physically Based Shading in Film and Game Production*, 2012.

[2] Khronos Group, "Vulkan guide: Dynamic rendering," https://docs.vulkan.org/guide/latest/dynamic_rendering.html, 2023, accessed: Mar. 2026.

[3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2022.

[4] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, W. Yifan, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi, T. Simon, C. Theobalt, M. Nießner, J. T. Barron, G. Wetzstein, M. Zollhöfer, and V. Golyanik, "Advances in neural rendering," *Computer Graphics Forum*, vol. 41, no. 2, pp. 703–735, 2022.

[5] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.

[6] Y. Chen, G. Chen, C. Wang *et al.*, "A survey on 3D Gaussian splatting," *arXiv preprint arXiv:2401.03890*, 2024.

[7] R. L. Cook and K. E. Torrance, "A reflectance model for computer graphics," *ACM Transactions on Graphics*, vol. 1, no. 1, pp. 7–24, 1982.